

大然机器人
智能一体化关节电机
CAN 通讯协议



DrEmpower



目录

第一章 简介.....	2
第二章 关节电机驱动板 CAN 通讯协议概要	3
2.1 标准控制 CAN 包（标准帧）组成.....	3
2.2 系统命令编号 order_num 和系统命令数据 order_flag.....	4
2.3 参数读取指令数据帧格式	4
2.4 参数设置指令及参数读取指令关节电机返回的数据帧格式	4
2.5 参数键码 Address 与数据类型 Data_type	4
第三章 CAN 通讯协议示例	6
3.1 控制指定编号关节电机转动到指定角度	6
3.2 控制指定关节电机以指定速度运行	7
3.3 设置关节电机 CAN_ID 号	8
3.4 函数 format_data().....	9
附录一 《指令对照表》	1
附录二 关节开放参数表.....	4



大然机器人

第一章 简介

1.1 简介

DrEmpower 智能一体化关节作为一款智能驱动器，不仅满足运动学控制的各项需求，同时也可满足动力学控制实践，使得用其开发的机器人产品更加贴近应用水平。

关节产品包括谐波减速器系列（HSA）、行星减速器系列（PDA）、中空系列（HCA）。具备运动智能特点，具备四大技能，包括运动控制、参数回读、参数设置和自主决策。

运动控制方面，除角度、转速、力矩控制外，还可进行运动助力控制、力位混合（自适应）控制、阻抗控制、急停控制。更多控制请见后续详情；

参数回读方面，可实时回读角度、转速、力矩。回读控制相关参数，如 PID 等。回读环境参数，如温度。更多参数请见后续详情；

参数设置方面，控制环中所有参数均可设置，也包括一些功能的使能设置。更多参数请见后续详情；

自主决策方面，包括碰撞感知、堵转/过流/温度保护、角度/转速/力矩限制等

DR TECH

大然机器人

第二章 关节电机驱动板 CAN 通讯协议概要

关节电机通过 CAN 组网时允许以并联方式连接到 CAN 上。网络中的每个关节电机均有独立的 CAN_ID 号（可自行设置）。同时所有关节电机有一个共同 CAN_ID 号，即 0 号（不可更改）。关节电机响应自己 ID 的指令，也会相应 0 号的指令。关节电机通讯使用 CAN 协议，默认 250Kbps 波特率。（还可支持 125 kbps，500 kbps，1Mbps，可自行设置）

硬件采用隔离式控制区域网络（CAN）物理层收发器，符合 ISO11898-2 标准的技术规范，可为 CAN 协议控制器和物理层总线分别提供差分接收和差分发送能力，信号传输速率最高可达 1Mbps，输出端可选配 120 欧终端电阻。

关节电机协议解析

2.1 标准控制 CAN 包（标准帧）组成

表 2-1 CAN 标准帧组成

含义	位置	7	6	5	4	3	2	1	0
帧信息	字节 01	FF	RTR	×	×	DLC			
帧 ID	字节 02	×	×	×	×		CAN_ID 高 3 位		
	字节 03	CAN_ID 低 3 位			CMD_ID				
数据帧	字节 04	data 1							
	字节 05	data 2							
	字节 06	data 3							
	字节 07	data 4							
	字节 08	data 5							
	字节 09	data 6							
	字节 10	data 7							
	字节 11	data 8							

表 2-2 CAN 标准帧解释

帧信息	第 7 位（FF）表示帧格式，在标准帧中，FF=0；第 6 位（RTR）表示帧的类型，RTR=0 表示为数据帧，RTR=1 表示为远程帧；DLC 表示在数据帧数据长度，在该 CAN 协议中帧数据永远为 64 位，即 DLC 永远为 8。
帧 ID	字节 02 低 3 位是 CAN_ID 的高 3 位，字节 03 的高 3 位是 CAN_ID 的低 3 位；字节 03 的低 5 位是 CMD_ID。
数据帧	64 位数据，靠近帧 ID 的为低位，末尾为最高位，符合 Intel 排序方法。如果指令中的数据为不足 64 位，则空余位用 0 补齐。

CAN 包中帧 ID 由 CAN_ID 和 CMD_ID 共同构成，CAN_ID 指定需要响应该条指令的关节电机，而 CMD_ID 指定其执行的命令。列如：一个关节电机的

ID 为 3，其会响应帧 ID 为 0b000011*****的 CAN 包（*代表任意值），而其具体执行的命令将根据*****决定。读取命令返回的帧 ID 将与发送的相同。

例如控制 3 号关节电机急停，则对应的帧 ID 计算过程如下：

(1) CAN_ID = 0x03;

(2) 查询《指令对照表》得到急停命令位于系统控制指令下，其 CMD_ID = 0x08;

(3) 将 CMD_ID 与(1)得到的 CAN_ID 相加可得

帧 ID = CAN_ID<<5 + CMD_ID = 0x60 + 0x008 = 0x68

(4) 将其变换为 16 位的帧 ID 0x0068，其中 0x68 为帧 ID 低位，0x00 为帧 ID 高位。

2.2 系统命令编号 order_num 和系统命令数据 order_flag

由于 CMD_ID 数量有限，关节电机功能却很多，因此在系统控制指令（CMD_ID=0x08）下需要在数据帧中增加两级命令编号，分别命名为加入系统命令编号 order_num 和系统命令数据 order_flag。order_num 占据 4 个字节，order_flag 占据 2 个字节。详见附录《指令对照表》。

继续以急停功能为例，查询《指令对照表》可知，其系统命令编号 order_num=0x06，系统命令数据 order_flag=0x00，则其完整 CAN 包为

帧信息	帧 ID 高位	帧 ID 低位	data1	data2	data3	data4	data5	data6	data7	data8
0x08	0x00	0x68	0x06	0x00	0x00	0x00	0x00	0x00	0x00	0x00
			order_num = 0x06 = 0x00000006				order_flag=0x00=0x0000			

2.3 参数读取指令（CMD_ID=0x1E）数据帧格式

data 1	data 2	data 3	data 4	data 5	data 6	data 7	data 8
参数键码<uint16>		类型代号<uint16>		0x00	0x00	0x00	0x00

2.4 参数设置指令（CMD_ID=0x1F）及参数读取指令关节电机返回的数据帧格式

data 1	data 2	data 3	data 4	data 5	data 6	data 7	data 8
参数键码<uint16>		类型代号<uint16>		参数值<数据类型和长度由 Data_type 确定，低位在前>			

2.5 参数键码 Address 与数据类型 Data_type

详见《关节开放参数表》，例如查表可得关节电机 CAN_ID 号(dr.config.can_id)对应的地址 Address = 31001、CAN 通信波特率(dr.can.config.baud_rate)对应的 Address = 21001。

Data_type: 数据类型，有以下 5 种数据类型，分别对应 0-4:

Data_type=0, 浮点数（float）,数据长度 32 位,符号‘f’;

Data_type=1, 无符号短整数 (uint16), 数据长度 16 位, 符号‘u16’;

Data_type=2, 有符号短整数 (int16), 数据长度 16 位, 符号‘s16’;

Data_type=3, 无符号短整数 (uint32), 数据长度 32 位, 符号‘u32’;

Data_type=4, 有符号短整数 (int32), 数据长度 32 位, 符号‘s32’;

示例：读取 4 号关节电机输出轴角度 (dr.output_shaft.angle)

帧信息： 0x08

帧 ID： CAN_ID 为 0x04, 向左移动 5 位空出 CMD_ID 位, 此时帧 ID 为 0x80, 查询《指令对照表》得到读取参数指令 CMD_ID=0x01E, 将 CMD_ID 与刚刚得到的帧 ID 相加得到 0x9E, 此时得到的数为 8 位, 将其变换为帧 ID 的 16 位 0x009E, 其中 9E 为帧 ID 低位, 00 位帧 ID 高位

帧数据： 查《指令对照表》可知, 读取参数指令数据帧有两个参数 Address 和 Data_type, 查询《关节开放参数表》表 dr.output_shaft.angle 对应的 Address = 38001 = 0x9471, 数据类型为 float32, 则 Data_type = 0 = 0x0000。

最后得到的 CAN 包从包头到包尾为:

帧信息	帧 ID 高位	帧 ID 低位	data 1	data 2	data 3	data 4	data 5	data 6	data 7	data 8
0x08	0x00	0x9E	0x71	0x94	0x00	0x00	0x00	0x00	0x00	0x00
			Address unsigned short		Data_type unsigned short					

关节电机返回的 CAN 包如下:

帧信息	ID 高位	ID 低位	data1	data2	data3	data4	data5	data6	data7	data8
0x08	0x00	0x9E	0x00	0x00	0x00	0x00	0x38	0xF0	0xB8	0xC2
							返回值 Value float32			

关节电机返回的 CAN 包中 data5~data8 为关节电机输出轴角度 (dr.output_shaft.angle), 其数据类型为 float32, 则先将数据取出, angle 为 0xC2B8F038, 经过转换后为-92.47 度, 至此通讯结束。

设置参数指令时帧数据和读取返回值是一致的, 首先查表确定其位于数据包的起始字节, 再将需要写入数据的每个字节倒序取出, 从数据帧起始字节低位开始放置, 例如目标值为 unsigned int 数 0x 324B550A, 起始字节为 4, 则数据包应该写为:

data1	data2	data3	data4	data5	data6	data7	data8
*	*	*	*	0x0A	0x55	0x4B	0x32

特别需要注意的是, signed int 负数以补码传输。

第三章 CAN 通讯协议示例

3.1 控制指定编号关节电机转动到指定角度

控制 CAN_ID 为 1 的关节电机以轨迹跟踪模式转动到 90°位置，同时指定角度输入滤波带宽为 10。

CAN 端口发送数据：

0x08 0x00 0x39 0x00 0x00 0xB4 0x42 0x00 0x00 0xE8 0x03

0x08 为数据头

0x00 0x39 的由来：

为 CAN_ID 与 CMD_ID 组合：1 的 16 进制为 0x01，向左移动 5 位空出 CMD_ID 位，此时帧 ID 为 0x20，查询《指令对照表》得到轨迹跟踪模式 CMD_ID 为 0x19，将 CMD_ID 与刚刚得到的帧 ID 相加得到 0x39，此时得到的数为 8 位，将其变换为帧 ID 的 16 位 0x0039，其中 39 为帧 ID 低位，00 为帧 ID 高位

0x00 0x00 0xB4 0x42 0x00 0x00 0xE8 0x03 的由来：

轨迹跟踪模式下有三个参数，分别为目标位置，目标速度(无作用)，角度输入滤波带宽。其中角度输入滤波带宽参数量纲为 0.01，则传输过程中需要换算为 10/0.01=1000。所以最终传输的三个数分别为 90,0,1000。

再利用库函数中的 `format_data([90,0,1000], 'f s16 s16', 'encode')` 转换得到最终的 16 进制：[0x00 0x00 0xB4 0x42 0x00 0x00 0xE8 0x03]。

(换算函数 `format_data` 见章末)

最终完整的 CAN 包为

帧信息	帧 ID 高位	帧 ID 低位	data1	data2	data3	data4	data5	data6	data7	data8
0x08	0x00	0x39	0x00	0x00	0xB4	0x42	0x00	0x00	0xE8	0x03
			目标位置： float32 量纲：1				目标速度： short int 量纲：0.01		角度输入滤波带宽： short int 量纲：0.01	

控制 1 号关节电机以梯形轨迹模式转动到-150 度位置，加速度为 5((r/min)/s)，最大速度为 10(r/min)。

CAN 端口发送数据：

0x08 0x00 0x3a 0x00 0x00 0x16 0xC3 0xE8 0x03 0xF4 0x01

0x08 为数据头

0x00 0x3a 为 CAN_ID 与 CMD_ID 组合：1 的 16 进制为 0x01，向左移动 5 位空出 CMD_ID 位，此时帧 ID 为 0x20，查询表格得到梯形轨迹模式的 CMD_ID

为 0x1A, 将 CMD_ID 与刚刚得到的帧 ID 相加得到 0x3a, 此时得到的数为 8 位, 将其变换为帧 ID 的 16 位 0x003a, 其中 3a 为帧 ID 低位, 00 为帧 ID 高位

0x00 0x00 0x16 0xC3 0xE8 0x03 0xF4 0x01 的由来:

梯形轨迹模式下有三个参数, 分别为目标位置, 目标速度及目标加速度。其中目标速度和目标加速度的量纲为 0.01, 则传输过程中需将速度换算为 $10/0.001=1000$, $5/0.001=500$ 。所以最终传输的三个数为-150,1000,500。

再利用库函数中的 `format_data9([-150,1000,500], 'f s16 s16', 'encode')` 转换为 16 进制: [0x00 0x00 0x16 0xC3 0xE8 0x03 0xF4 0x01]。

最终完整的 CAN 包为

帧信息	帧 ID 高位	帧 ID 低位	data1	data2	data3	data4	data5	data6	data7	data8
0x08	0x00	0x3a	0x00	0x00	0x16	0xC3	0xE8	0x03	0xF4	0x01
			目标位置: float32 量纲: 1				目标速度: short int 量纲: 0.01		目标加速度: short int 量纲: 0.01	

3.2 控制指定关节电机以指定速度运行

控制 1 号关节电机以速度前馈模式以速度 20(r/min)转动, 前馈扭矩为 1.5Nm

CAN 端口发送数据:

0x08 0x00 0x3c 0x00 0x0 0xA0 0x41 0x96 0x00 0x01 0x00

0x08 为数据头

0x00 0x3c 为 CAN_ID 与 CMD_ID 组合: 1 的 16 进制为 0x01, 向左移动 5 位空出 CMD_ID 位, 此时帧 ID 为 0x20, 查询表格得到关节电机速度指令消息的 CMD_ID 为 0x01c, 将 CMD_ID 与刚刚得到的帧 ID 相加得到 0x3c, 此时得到的数为 8 位, 将其变换为帧 ID 的 16 位 0x003c, 其中 3c 为帧 ID 低位, 00 为帧 ID 高位

0x00 0x0 0xA0 0x41 0x96 0x00 0x01 0x00 的由来:

速度控制指令有三个参数, 分别为目标速度, 模式参数及目标模式。其中速度前馈模式对应的目标模式为 1, 对应的模式参数为前馈扭矩, 量纲为 0.01, 则传输过程中需将前馈扭矩换算为 $1.5/0.001=150$ 。

再利用库函数中的 `format_data([20,150,1], 'f s16 u16', 'encode')` 转换为 16 进制: [0x00 0x00 0xA0 0x41 0x96 0x00 0x01 0x00]。

最终完整的 CAN 包为

帧信息	帧 ID 高位	帧 ID 低位	data1	data2	data3	data4	data5	data6	data7	data8
0x08	0x00	0x3c	0x00	0x00	0xA0	0x41	0x96	0x00	0x01	0x00
			目标速度: float32				前馈扭矩: short int		目标模式: unsigned short int	

			量纲：1	量纲：0.01	量纲：1
--	--	--	------	---------	------

控制 1 号关节电机以速度爬升模式以速度爬升速率(加速度)5((r/min)/s)加速至 20(r/s)转动

CAN 端口发送数据：

0x08 0x00 0x3C 0x00 0x00 0xA0 0x41 0xf4 0x01 0x02 0x00

0x08 为数据头

0x00 0x3C 为 CAN_ID 与 CMD_ID 组合：1 的 16 进制为 0x01，向左移动 5 位空出 CMD_ID 位，此时帧 ID 为 0x20，查询表格得到关节电机速度控制指令 CMD_ID 为 0x01c，将 CMD_ID 与刚刚得到的帧 ID 相加得到 0x3c，此时得到的数为 8 位，将其变换为帧 ID 的 16 位 0x003c，其中 3c 为帧 ID 低位，00 为帧 ID 高位

0x00 0x00 0xA0 0x41 0xf4 0x01 0x02 0x00 的由来：

速度控制指令有三个参数，分别为目标速度，模式参数及目标模式。其中速度爬升模式对应的目标模式为 2，对应的模式参数为速度爬升速率，量纲为 0.01，则传输过程中需将速度爬升速率换算为 5/0.001=500。

再利用库函数中的 format_data([20,500,2], 'f s16 u16', 'encode')转换为 16 进制：[0x00 0x00 0xA0 0x41 0xf4 0x01 0x02 0x00]。

最终完整的 CAN 包为

帧信息	帧 ID 高位	帧 ID 低位	data1	data2	data3	data4	data5	data6	data7	data8
0x08	0x00	0x3c	0x00	0x00	0xA0	0x41	0xF4	0x01	0x02	0x00
			目标速度： float32 量纲：1				速度爬升速率： short int 量纲：0.01		目标模式： unsigned short int 量纲：1	

3.3 设置关节电机 CAN_ID 号

设置关节电机 CAN_ID 号，将 1 号关节电机的 CAN_ID 号(dr.config.can_id)修改为 5

CAN 端口发送数据：

0x08 0x00 0x3f 0x19 0x79 0x03 0x00 0x05 0x00 0x00 0x00

0x08 为数据头

0x00 0x3f 为 CAN_ID 与 CMD_ID 组合：关节电机 CAN_ID 号为 1，向左移动 5 位空出 CMD_ID 位，此时帧 ID 为 0x20，查询《指令对照表》得到参数设置指令的 CMD_ID 为 0x01f，将 CMD_ID 与刚刚得到的帧 ID 相加得到 0x3f，此

时得到的数为 8 位，将其变换为帧 ID 的 16 位 0x003f，其中 3f 为帧 ID 低位，00 为帧 ID 高位

0x19 0x79 0x03 0x00 0x05 0x00 0x00 0x00 的由来：

查询《关节开放参数表》表 dr.config.can_id 对应的 Address = 31001 = 0x7919，数据类型为 uint32，则 Data_type = 3 = 0x0003。参数设置指令的第三个参数为目标值 Value，这里 Value=5。

再利用库函数中的 format_data([31001,3,5], 'u16 u16 u32', 'encode') 转换为 16 进制 [0x19 0x79 0x03 0x00 0x05 0x00 0x00 0x00]。

最终完整的 CAN 为

帧信息	帧 ID 高位	帧 ID 低位	data1	data2	data3	data4	data5	data6	data7	data8
0x08	0x00	0x3F	0x19	0x79	0x03	0x00	0x05	0x00	0x00	0x00
			Address u16		Data_type u16		Value u32			

3.4 函数 format_data()

函数使用说明：

该函数位于 **yf_14_can.py** 库函数文件中；

函数原型：def format_data(data=[], format="f f", type='decode')

调用函数时：

若使用数据转为 **byte 类型**，在 data 处写入 float, int, unsigned int, short, unsigned short, 用逗号“,”隔开，在 format 处输入对应类型名：'f, s32, u32, s16, u16'，多个数据时用空格“ ”隔开，在 type 处输入'encode'；（**建议只输入两个数据**）

若使用函数将 **byte 类型转为普通数据类型**，在 data 处输入二进制数，用逗号“,”隔开，在 format 处输入需要转化的类型的对应名，分别为：'f, s32, u32, s16, u16'，多个数据时，用空格“ ”隔开，在 type 处输入'decode'。（**data 只能输入八位**）

其中，s16、u16 只占用两位 byte 位，f、s32、u32 占用四位。

附录一 《指令对照表》

- (1) 量纲非 1 时，参数需要先除以量纲再装入数据帧
- (2) 数据类型代号：float (0); uint16 (1); int16 (2); uint32 (3); int32 (4)
- (3) 起始字节为参数在数据帧中的起始字节位置，参考库函数阅读下表效果更佳
- (4) 表格中“对应功能”名称用库函数名称代替
- (5) 函数库中读取参数和修改参数均基于 CMD_ID=0x1E 和 CMD_ID=0x1F 改写，使用通信协议时可参照
- (6) CMD_ID=0x08 系统控制指令中大部分功能需要先执行 CMD_ID=0x06 和 CMD_ID=0x0C 预设参数指令，具体可参考对应库函数

CMD_ID	对应功能	参数名称	单位	起始字节	数据类型	量纲	备注
0x19	set_angle mode=0	目标角度	°	0	float32	1	
		转速	r/min	4	uint16	0.01	
		输入滤波带宽		6	uint16	0.01	
0x1A	set_angle mode=1	目标角度	°	0	float32	1	
		转速	r/min	4	uint16	0.01	
		角加速度	r/min/s	6	uint16	0.01	
0x1B	set_angle mode=2	目标角度	°	0	float32	1	
		转速	r/min	4	uint16	0.01	
		力矩	Nm	6	uint16	0.01	
0x1C	set_speed	目标转速	r/min	0	float32	1	
		加速度	r/min/s	4	uint16	0.01	
		模式		6	uint16	1	
0x1D	set_torque	目标力矩	Nm	0	float32	1	
		加力矩	Nm/s	4	uint16	0.01	
		模式		6	uint16	1	
0x1E	read_property	参数键码		0	uint16	1	
		数据类型代号		2	uint16	1	
0x1F	write_property	参数键码		0	uint16	1	见《开放参数表》

CMD_ID	对应功能	参数名称	单位	起始字节	数据类型	量纲	备注		
		数据类型代号		2	uint16	1			
		参数值		4	由类型代号确定	1			
0x06	预设 4 个参数	参数 1		0	int16	0.01	用于 motion_aid_multi		
		参数 2		2	uint16	0.01			
		参数 3		4	uint16	0.01			
		参数 4		6	int16	0.01			
0x08	系统控制指令	order_num		0	uint32				
		order_flag		4	uint16				
		系统控制指令对照表							
		order_num	order_flag		对应功能		备注		
		0x01	--		save config				
		0x03	--		reboot				
		0x05	--		set zero position				
		0x06	--		estop				
		0x10	0x00		set angles mode=0		须先执行 0x0C		
			0x01		step_angle/step_angles mode=0				
		0x11	0x00		set_angles mode=1				
			0x01		step_angle mode=1				
			0x02		step_angles mode=1				
			0x03		set_angles adaptive				
		0x12	0x04		motion aid multi		须先执行 0x06		
			0x00		set_angles mode=2		须先执行 0x0C		
		0x01		step_angle/ step_angles mode=2					
		0x13	--		set speeds				
		0x14	--		set torques				
		0x15	kp/0.001（16 位）	kd/0.001（16 位）	impedance_control				
		0x16	kp/0.001（16 位）	kd/0.001（16 位）	impedance_control_multi 预备				
		0x17	--		impedance_control_multi 启动				
		0x18	--		set speed limit				
		0x19	--		set torque limit				
		0x20	--		set speed adaptive				
		0x21	--		set torque_adaptive				
		0x23	--		set zero position temp				
		0x24	--		get_angle_speed_torque				
		0x0B	set angle adaptive	角度	°	0	float32	1	

CMD_ID	对应功能	参数名称	单位	起始字节	数据类型	量纲	备注
		转速	r/min	4	uint16	0.01	
		力矩	Nm	6	uint16	0.01	
0x0C	预设三个参数	参数 1			float32	1	多个关节电机控制时预先使用
		参数 2			int16	0.01	
		参数 3			int16	0.01	
0x0D	motion_aid	角度	°	0	int16	0.01	转速由随其后的 set_speed_adaptive 提供
		角度差	°	2	uint16	0.01	
		转速差	r/min	4	uint16	0.01	
		力矩	Nm	6	int16	0.01	
0x0E	init_config	id		0	uint32	1	



附录二 关节开放参数表

注，对于以下参数：

- (1) 读写列 1 表示可读可写，0 表示只读；
- (2) 名称中带 config 的参数均可由 save_config() 函数保存并在重启后依然有效；
- (3) 使用 read_property() 函数读取；
- (4) 使用 write_property() 函数改写
- (5) 数据类型代号：float (0)；uint16 (1)；int16 (2)；uint32 (3)；int32 (4)。

序号	含义	名称	单位	默认值	类型	键码	读写	备注
1	电压	dr.voltage	V		float	1	0	
2	电流	dr.i	A		float	2	0	
3	CAN 总线波特率	dr.can.config.baud_rate	bit/s	1M	uint32	21001	1	
4	是否开启 CAN 实时状态反馈	dr.can.enable_state_feedback		0	uint32	22001	1	1 表示开启，0 表示不开启，版本号 240627 及以后不可由 save_config() 函数保存
5	固件版本日期	dr.version_date			uint32	30004	0	固件版本号为 231207 及以后版本可用
6	CAN 总线 ID 号	dr.config.can_id		7	uint32	31001	1	取值范围：1~63
7	CAN 总线实时状态反馈时间间隔	dr.config.state_feedback_rate_ms	ms	2	uint32	31002	1	比如设置 3ms，则每隔 3ms 反馈一次状态
8	关节型号	dr.config.product_model			uint32	31003	0	参照手册中的关节型号代码
9	是否开启角度限制属性	dr.config.enable_angle_limit		0	uint32	31201	1	1 表示开启，0 表示不开启
10	最小角度限位	dr.config.angle_min	°	-180	float	31202	1	
11	最大角度限位	dr.config.angle_max	°	180	float	31203	1	
12	减速比	dr.config.gear_ratio			float	31204	0	
13	堵转电流	dr.config.stall_current_limit	A	50	float	31205	1	
14	是否开启碰撞检测	dr.config.enable_crash_detect		1	uint32	31206	1	

附录 关节开放参数表

序号	含义	名称	单位	默认值	类型	键码	读写	备注
15	碰撞检测灵敏度	dr.config.crash_detect_sensitivity		150	float	31207	1	该值需大于 0，越小越灵敏
16	是否开启多圈计数角度限制	dr.config.enable_encoder_circular_limit		1	uint32	31208	1	超出该限制关节将无法记住重启前的角度，需重新设置零点。若关闭该限制则必将编码器供电线拔掉，即默认系统不需要关机后零点位置。该参数只针对具有多圈计数功能关节有效。
17	是否运动到指定位置	dr.controller.position_done		1	uint32	32002	0	为 1 表示到达指定位置，为 0 则未到达
18	判定运动到位的定位精度	dr.controller.position_precision	°	0.036	float	32003	1	固件版本号为 231207 及以后版本可用
19	是否启用转速限制	dr.controller.config.enable_speed_limit		1	uint32	32101	1	1 表示启用，0 表示不启用
20	位置增益 P	dr.controller.config.angle_gain			float	32102	1	
21	转速增益 D	dr.controller.config.speed_gain			float	32103	1	
22	积分增益 I	dr.controller.config.speed_integrator_gain			float	32104	1	
23	最大限制转速	dr.controller.config.speed_limit	r/min	+∞	float	32105	1	该值针对电机，输出端需乘以减速比
24	超速容忍度	dr.controller.config.speed_limit_tolerance	%	1.2	float	32106	1	比如设为 1.2 则超出 20%后才会报错
25	负载转动惯量	dr.controller.config.inertia		0	float	32107	1	默认为 0
26	输入滤波带宽	dr.controller.config.input_filter_bandwidth		2	float	32108	1	
27	电机极对数	dr.motor.config.pole_pairs			int32	33101	0	
28	电机相电感	dr.motor.config.phase_inductance			float	33102	0	
29	电机相电阻	dr.motor.config.phase_resistance			float	33103	0	
30	电机力矩常数	dr.motor.config.torque_constant			float	33104	0	
31	最大限制电流	dr.motor.config.current_limit			float	33105	1	
32	过流容忍度	dr.motor.config.current_limit_margin	A		float	33106	1	比如设为 1.2 则超出 1.2A 后才会报错
33	最大力矩限制值	dr.motor.config.torque_limit	Nm	+∞	float	33107	1	该值对象是电机，输出端需除以减速比
34	电流控制带宽	dr.motor.config.current_control_bandwidth			float	33108	1	
35	FOC Q 轴电流	dr.motor.Iq_measured	A		float	33201	0	

附录 关节开放参数表

序号	含义	名称	单位	默认值	类型	键码	读写	备注
36	FOC D 轴电流	dr.motor.Id_measured	A		float	33202	0	
37	设置零点后输出轴编码器零位值	dr.encoder.pos_zero_output			int32	34101	0	中空关节专属
38	刚启动时读取到的输出轴编码器值	dr.encoder.abs_pos_power_on			int32	34102	0	中空关节专属
39	绝对位置误差的前半个周期长度	dr.encoder.first_half_T			float	34103	1	中空关节专属，标定后误差补偿使用
40	绝对位置误差的前半个周期幅值	dr.encoder.first_half_M			float	34104	1	中空关节专属，标定后误差补偿使用
41	绝对位置误差的后半个周期幅值	dr.encoder.second_half_M			float	34105	1	中空关节专属，标定后误差补偿使用
42	绝对位置误差截距	dr.encoder.intercept			float	34106	1	中空关节专属，标定后误差补偿使用
43	绝对位置误差前半个周期幅值是否向上	dr.encoder.first_half_up			uint32	34107	1	中空关节专属，标定后误差补偿使用
44	驱动板温度	dr.board_temperature	°C		float	36002	0	
45	是否开启驱动板温度保护	dr.board_temperature.config.enabled		1	uint32	36101	1	
46	驱动板温度保护下限	dr.board_temperature.config.temp_limit_lower	°C	100	float	36102	1	
47	驱动板温度保护上限	dr.board_temperature.config.temp_limit_upper	°C	120	float	36103	1	
48	电机温度	dr.motor_temperature	°C		float	37002	0	
49	是否开启电机温度保护	dr.motor_temperature.config.enabled		1	uint32	37101	1	
50	电机温度保护下限	dr.motor_temperature.config.temp_limit_lower	°C	80	float	37102	1	
51	电机温度保护上限	dr.motor_temperature.config.temp_limit_upper	°C	100	float	37103	1	
52	输出轴角度	dr.output_shaft.angle	°		float	38001	0	
53	输出轴转速	dr.output_shaft.speed	r/min		float	38002	0	
54	输出轴力矩	dr.output_shaft.torque	Nm		float	38003	0	
55	输出轴最小临时角度限位	dr.output_shaft.angle_min	°	-180	float	38004	1	
56	输出轴最大临时角度限位	dr.output_shaft.angle_max	°	180	float	38005	1	
57	是否开启临时角度限位	dr.output_shaft.enable_angle_limit		0	uint32	38006	1	

附录 关节开放参数表